Python Course — Day 52 Reference

Flask API Design and REST Principles | 5 June 2026

SESSION 1 — REST API Structure

Lesson 1 — HTTP Methods

```
# HTTP methods — use correctly GET — retrieve data — never changes data POST — create new record PUT — replace full record PATCH — partial update — only what changed DELETE — remove record # URLs — nouns not verbs /drivers # GET all, POST create /drivers/1 # GET one, PATCH update, DELETE remove # Wrong — verbs in URLs /getDrivers /createDriver /deleteDriver # Correct — nouns only /drivers /drivers/1
```

Lesson 2 — Status Codes

```
200 — OK — request succeeded 201 — Created — new record saved 400 — Bad Request — validation failed 401 — Unauthorized — not logged in 403 — Forbidden — logged in but no permission 404 — Not Found — record does not exist 500 — Server Error — something broke in your code
```

Lesson 3 — request.get_json() vs request.form

```
request.form.get('username') # reads HTML form submissions request.get_json() # reads JSON from JavaScript or Postman data = request.get_json() username = data.get('username') # reads key from JSON body
```

Business Use Case — Session 1

REST is the agreed standard the whole industry uses. Any frontend — mobile, web, desktop — can talk to your Flask backend using the same rules.

SESSION 2 — Routes Connected to Database

Lesson 1 — GET All Records

```
@app.route('/drivers', methods=['GET']) def get_drivers(): conn = get_db() cursor = conn.cursor() cursor.execute('SELECT * FROM drivers') drivers = [dict(row) for row in cursor.fetchall()] conn.close() return jsonify({'drivers': drivers}), 200
```

Lesson 2 — GET One with 404

```
@app.route('/drivers/', methods=['GET']) def get_driver(driver_id): conn = get_db() cursor = conn.cursor() cursor.execute('SELECT * FROM drivers WHERE id = ?', (driver_id,)) driver = cursor.fetchone() conn.close() if not driver: # record doesn't exist return jsonify({'error': 'Driver not found'}), 404 return jsonify(dict(driver)), 200
```

Lesson 3 — POST Create

```
@app.route('/drivers', methods=['POST']) def create_driver(): data = request.get_json() # read JSON body name = data.get('name') phone = data.get('phone') license_number = data.get('license_number') status = data.get('status', 'active') # default to active if not name or not phone or not license_number: return jsonify({'error': 'Name, phone and license required'}), 400 cursor.execute('INSERT INTO drivers (name, phone, license_number, status) VALUES (?,?,?,?)', (name, phone, license_number, status)) conn.commit() new_id = cursor.lastrowid # id of newly created record return jsonify({'message': 'Driver created', 'id': new_id}), 201
```

Lesson 4 — DELETE

```
# Always check exists before deleting if not driver: return jsonify({'error': 'Driver not found'}), 404 cursor.execute('DELETE FROM drivers WHERE id = ?', (driver_id,)) conn.commit() return jsonify({'message': f'Driver {driver_id} deleted'}), 200
```

Business Use Case — Session 2

A logistics company manages 50 drivers. Admin adds a driver — POST. Dashboard loads all — GET. Driver suspended — DELETE. Every operation hits the database correctly with proper status codes.

SESSION 3 — PATCH Partial Update

Lesson 1 — PUT vs PATCH

```
# PUT — replace entire record — send all fields PUT /drivers/1 {"name": "Thabo", "phone": "0799999999", "license_number": "GP123", "status": "active"} # PATCH — update only what changed — send only changed fields PATCH /drivers/1 {"phone": "0799999999"} # only phone changes, everything else stays
```

Lesson 2 — PATCH Route

```
@app.route('/drivers/', methods=['PATCH']) def update_driver(driver_id): conn = get_db() cursor = conn.cursor() cursor.execute('SELECT * FROM drivers WHERE id = ?', (driver_id,)) driver = cursor.fetchone() if not driver: conn.close() return jsonify({'error': 'Driver not found'}), 404 data = request.get_json() name = data.get('name', driver['name']) # new value or keep existing phone = data.get('phone', driver['phone']) license_number = data.get('license_number', driver['license_number']) status = data.get('status', driver['status']) cursor.execute(""" UPDATE drivers SET name=?, phone=?, license_number=?, status=? WHERE id=? """, (name, phone, license_number, status, driver_id)) conn.commit() conn.close() return jsonify({'message': f'Driver {driver_id} updated'}), 200
```

Business Use Case — Session 3

A driver changes their phone number. Admin sends PATCH with only the new phone — everything else stays unchanged. Without PATCH you'd send all fields just to change one.

KEY RULES — DAY 52

- GET never changes data — only retrieves
- POST creates — returns 201 Created
- PATCH updates partially — use data.get('field', existing_value) to keep unchanged fields
- PUT replaces entirely — all fields must be sent
- DELETE removes — always check exists first, return 404 if not found
- Always check if record exists before UPDATE or DELETE
- request.get_json() reads JSON body — request.form reads HTML form
- cursor.lastrowid gets the id of the last inserted record
- URLs use nouns not verbs — /drivers not /getDrivers
- Return correct status codes on every route — 200, 201, 400, 404
- conn.commit() after INSERT/UPDATE/DELETE — conn.close() always last
- GitHub push after every session: git add . → git commit → git push origin main

---